

```

import os
from datetime import datetime, timedelta
from jose import jwt
from passlib.context import CryptContext
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer

SECRET_KEY = os.getenv("JWT_SECRET", "dev-secret-change-in-production")
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 60

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/auth/login")

def hash_password(password: str) -> str:
    return pwd_context.hash(password)

def verify_password(plain: str, hashed: str) -> bool:
    return pwd_context.verify(plain, hashed)

def create_access_token(user_id: str, tenant_id: str, role: str) -> str:
    """
    We embed tenant_id in the JWT itself. That way, once a user logs in,
    every subsequent request carries proof of which tenant they belong to
    -- we don't have to trust a client-supplied tenant_id (which could be
    spoofed). The token is signed, so it can't be tampered with.
    """
    expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    payload = {
        "sub": user_id,
        "tenant_id": tenant_id,
        "role": role,
        "exp": expire,
    }
    return jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM)

def decode_access_token(token: str) -> dict:
    try:
        return jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])

```

```
except jwt.JWTError:
    raise HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Invalid or expired token",
    )

def get_current_user(token: str = Depends(oauth2_scheme)) -> dict:
    """Dependency injected into protected routes to identify the caller."""
    payload = decode_access_token(token)
    return {
        "user_id": payload["sub"],
        "tenant_id": payload["tenant_id"],
        "role": payload["role"],
    }
```